

Where we stand — SIH26055, as of 2026-09-11

Read this first if you are picking the project up, building the deck, or scoping the edge lane. It is a pointer document: every number here is quoted from an artefact on disk and says where that artefact is. Nothing is asserted here that is not measured somewhere else in this repository.

Authority is unchanged: EVALUATION.md defines the metrics, DECISIONS.md records the decisions, this file only says where the project is. If this file and either of those disagree, they win.

1. The one-paragraph answer

We built an RF scan-scheduling environment from the Turing Synthetic Radar Dataset, validated it against four gates, built a seven-rung ladder of baselines from random through a published adaptive strategy, and trained a recurrent RL scheduler that **beats every rung on the problem statement's own joint objective**. The best model Pareto-dominates the strongest simple heuristic on all four scheduler metrics simultaneously, and it was chosen by a selection rule fixed in code before the data that confirmed it was ever generated.

2. The headline number

Rung 17c — Recurrent PPO (LSTM), reward reward_balance, training seed 2, 300k of 400k steps. Measured over **5,643 episodes** (33 rungs × 57 scenarios × 3 seeds), runs/final_2026-09-11/.

metric (EVALUATION.md §4)	rung 17c	rung 5 recency (the bar)	rung 2 round_robin (the floor)
Interception ratio ↑	0.1304	0.1105	0.0605
Censored mean intercept time ↓	3.04 s	3.34 s	4.18 s
Emitter coverage ↑	0.9019	0.8874	0.8650
Average intercept rate ↑	1.191 /s	1.153 /s	1.118 /s

Paired, per episode (same scenario, same seed, same truth grid, n = 171):

	wins on ratio	wins on intercept time	wins on both
17c vs round_robin	89.5%	85.4%	81.9%
17c vs recency	77.8%	68.4%	54.4%

both is the column that matters — D14 measured that no trivial strategy is good at both objectives, so Pareto-dominating the reference is the bar and winning one column alone is not evidence.

3. The five things worth putting in a deck

Each of these is a *finding*, not a claim — with the artefact behind it.

(1) The bar was never round-robin, and saying so is the interesting part. The problem statement targets open-loop round-robin. But a one-line index policy (`argmax(hit rate + gap in sweeps)`, rung 5) beats round-robin on the joint metric **67.3%** of the time. We made *that* the bar instead. Beating round-robin is the minimum, not a result. → D46, D71

(2) A published adaptive strategy fails the PS's own joint objective. Apfeld et al.'s method (rung 6) posts the second-highest interception ratio in the whole table (0.2455) — and recency beats it on the joint metric **97.1%** of the time, because it buys that ratio with 14.86 s intercept time and 0.367 coverage. It camps. This is the single clearest argument for why two metrics are reported together and never one. → D44, D45, D71

(3) Our RL scheduler beats every rung on the ladder, including the literature. All 24 live checkpoints beat the floor decisively (50.9–81.9% joint), median **35.4%** against the bar, best **54.4%**. 23 of 24 beat `apfeld_active_rfs`, the strongest published rung on the joint objective. → D71

(4) The winner was picked by a rule written before the data existed. `rfenv/selection.py` fixed the checkpoint-selection criterion in code — highest $P(\text{dominates rung 5}) - P(\text{dominated by rung 5})$ on a *validation half whose emitters the training pool never contained* — and it picked rung 17c on validation alone. 17c then scored highest of all 24 on the full development set. The rule never saw the data it turned out to be right about. A checkpoint chosen after seeing the results table would carry no weight (same logic as D39's gate thresholds). → D61, D71

(5) We found and killed four of our own defects, each with the number that exposed it. Judges reward this and most decks cannot produce it. - **D52** — the reward's exploration term read staleness *inverted*: it paid most for revisiting the band just left (+0.419 vs −0.0025 for a band untouched 500 slots). - **D53** — the camping penalty reset on any action change, so a 2-band ping-pong paid exactly what a full sweep paid; it ranked ping-pong (coverage 0.261) above round-robin (0.921). - **D54** — inference took the `argmax` of a policy that had *not* collapsed (entropy 2.369 against $\ln 36 = 3.584$); sampling the same checkpoint visits 31 of 36 bands and triples coverage. - **D56** — a measurement substituted a hand-written `step % 36` sweep for the registered rung 2 and inverted its own conclusion. A test now fails the build on any hand-rolled band cycle (D69).

4. What is NOT proven — state these honestly

- **Seed variance exceeds every reward-arm effect we have argued about.** The 24 checkpoints span 18.7–54.4% against the bar. The control/treatment gap D65 and D68 debated is a few points inside that. No single-checkpoint comparison between the two rewards is trustworthy at this sample size. → D71 finding 4
 - **The held-out set has never been touched.** 45 scan/stare test pairs, fetched by a rule fixed in advance, untouched through the entire project (D8). Every number in this repository is a *development-set* number. That is the correct state — they are spent once, on a final system.
 - **Gate 1 is MEASURED, not PASSED.** D37 fixed its convention and deliberately left its threshold undecided. Gates 2, 3, 4 pass. → `EVALUATION.md` §6
 - **What the gates cannot detect is written down.** Gate 2's 0.000 pp is algebraically forced; gate 3's reference is co-parameterised with the environment; **no gate covers the ±500 MHz band half-width.** → D42. Read it before quoting any gate figure.
 - **D30 (AoA / PulseWidth in the observation) is still OPEN.** Never addressed by the RL lane.
 - **26 of 56 trained checkpoints can never be run again** — killed by observation changes (D49, D55, D67). Each change was paid deliberately; the cost is real and is documented. → `MODEL_COMPARISON.md` §1
-

5. For the edge lane

Grounding and boundaries are in `EDGE_LANE_HANDOFF.md` (regenerated 2026-09-11, current). The facts that constrain an edge design:

Action	one of 36 bands per decision
Observation	183 floats ($36 \times 5 + 3$), all derived from the agent's own scan history
Decision cadence	one per dwell — 1 or 2 slots , 50 ms per slot
Episode	600 slots = 30 s
Policy	Recurrent PPO, <code>MlpLstmPolicy</code> , SB3 defaults; checkpoint ≈ 10 MB on disk
Inference	samples the policy, does not take the argmax (D54) — this changes deployed behaviour and is not a detail
Observation is not normalised to [0,1]	two blocks declare ceilings above 1.0 so 1.0 <i>means</i> something (D55) — any reimplementaion must reproduce the scaling exactly

The deployability constraint is enforced, not trusted. `base_lines/guard.py` strips `info` to five observable keys before any policy sees it, so a scheduler reaching for truth-side state raises `KeyError` in a test rather than quietly scoring well. Anything ported to an edge target has to honour the same boundary. → D19, D20, D29, D34

Open, and genuinely useful to scope: `PHASE_SWITCH_FUTURE_WORK.md` carries the two directions still worth trying after the hard explore/exploit gate was built, measured as *worse than the camper*, and removed the same day (D66). `THREAT_WEIGHTING_BRIEF.md` (D70) covers taking a threat priority from outside the scheduler — the PS names this and nothing implements it yet.

6. Where everything lives

Want	Go to
What a metric means	<code>EVALUATION.md</code> §4
The full ladder table, latest run	<code>runs/final_2026-09-11/comparison.md</code> , and <code>EVALUATION.md</code> §5
Why any choice was made	<code>DECISIONS.md</code> — D71 is the newest, D1 the oldest
Every trained model, its width, what it measured	<code>MODEL_COMPARISON.md</code>
Every training attempt, including failures	<code>ITERATION_LEDGER.md</code>

Want	Go to
The environment's buildable spec	<code>ENVIRONMENT_SPEC.md</code>
State/action formulation for the RL lane	<code>STATE_ACTION_FORMULATION.md</code>
Is this tree safe to hand over	<code>python scripts/doctor.py</code>
Reproduce the headline	<code>python -m rfenv.compare --seeds 3 --sampled 10 --figures</code>
Re-screen the rewards (minutes, no GPU)	<code>python -m rfenv.reward_gate</code>

Suite status 2026-09-11: 433 passed / 144 skipped / 0 failed with the training stack; 261 / 290 / 0 without it. The training stack is deliberately optional — a module-level import of it in a test aborts collection of the *whole* suite, which has happened twice.